S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 11 Post lab

Aim: To design and simulate an optimized elevator scheduling system using the SCAN algorithm for a smart building with multiple elevators and a single request button per floor, minimizing total movement and wait time.

Name: Pranav Mohan Paunikar

USN: CM24032

Semester / Year: IV/II

Academic Session: 2025-26

Date of Performance:

Date of Submission:

Aim: To design and simulate an optimized elevator scheduling system using the SCAN algorithm for a smart building with multiple elevators and a single request button per floor, minimizing total movement and wait time.

Objective:

- **To understand the working of the SCAN scheduling algorithm**
- **To simulate elevator movement based on floor requests**
- **To minimize total floor movement (seek time)**
- **To analyze performance for different directions (UP/DOWN)**
- **To extend the concept to multi-elevator systems**

Requirement:

- **Hardware Requirements**
  - **Computer system with minimum 4GB RAM**
- **Software Requirements**
  - **C Compiler (e.g., Turbo C / GCC)**
  - **Operating System: Windows / Linux**

Theory:

1. Introduction

The SCAN (Elevator) algorithm is a disk scheduling algorithm adapted for elevator systems in smart buildings. It moves the elevator in one direction, servicing all pending requests along its path, then reverses direction at the last request (or building extreme) and continues. This ensures fairness, prevents starvation, and minimizes mechanical wear by reducing unnecessary direction changes.

2. Key Concepts

1. **Total Movement:** The sum of floors traversed by the elevator car while servicing all requests. It directly impacts energy consumption and passenger wait time.
2. **Directional Sweep:** The elevator maintains a current direction (UP or DOWN) and services requests in that order until no requests remain ahead, then reverses.

3. **Zone Partitioning:** In multi-elevator systems, the building is divided into zones. Each elevator services requests only within its assigned zone, reducing overall movement and enabling parallel processing.

3. SCAN Algorithm Steps

1. Start at initial position with a defined direction
2. Move in current direction, servicing all requests encountered
3. Continue until reaching the last request in that direction (or building end)
4. Reverse direction
5. Repeat until all requests are serviced

4. Advantages of SCAN

- **Fairness:** Every request gets serviced eventually
- **Efficiency:** Batching requests in one direction reduces travel
- **Predictability:** Movement pattern is deterministic

5. Multi-Elevator Optimization

For dual-head systems, the building is partitioned at the midpoint:

- **Head 1 (Lower Zone):** Services floors 0 to midpoint, sweeps downward
- **Head 2 (Upper Zone):** Services floors midpoint+1 to top, sweeps upward

This prevents elevator clustering and reduces average seek distance.

Example: Single-Head vs Dual-Head SCAN

Problem Statement

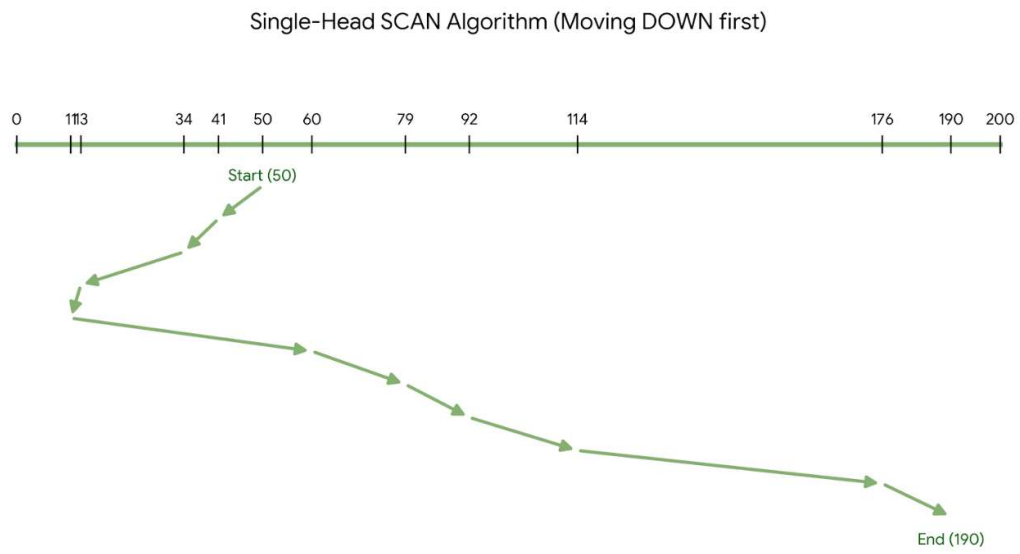
A building has floors numbered 0 to 200. Requests are at floors: **176, 79, 34, 60, 92, 11, 41, 114, 13, 190**

Initial Positions:

- Single-head system: Starting at floor 50
- Dual-head system: Head 1 at 50 (lower zone), Head 2 at 150 (upper zone)

Calculate total movement for each system.

- **Solution A: Single-Head SCAN (Moving DOWN first)**



Step 1: Sort requests in ascending order: 11, 13, 34, 41, 60, 79, 92, 114, 176, 190

Step 2: Starting at 50, moving DOWN to lowest request:

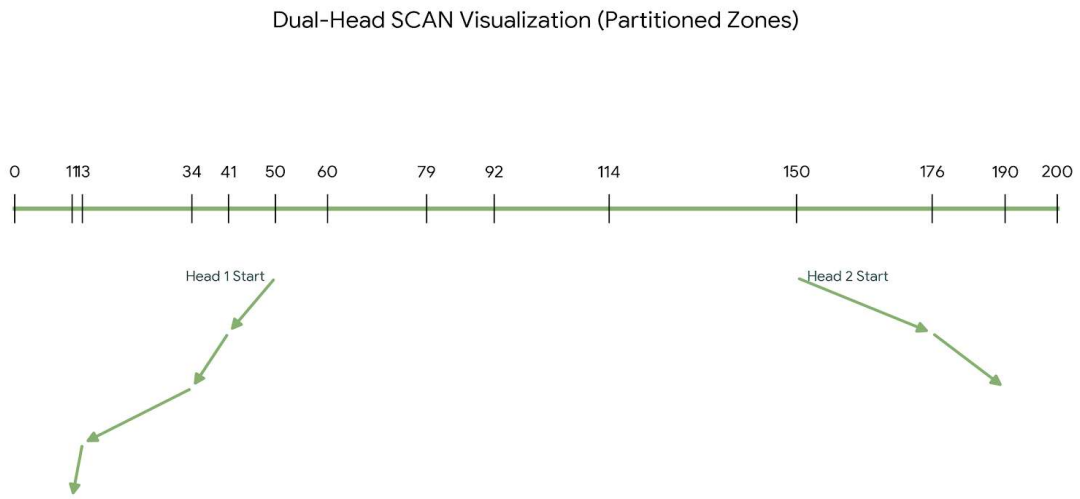
From	To	Movement
50	41	9
41	34	7
34	13	21
13	11	2

Step 3: Reverse direction, move UP to highest request:

From	To	Movement
11	60	49
60	79	19
79	92	13
92	114	22
114	176	62
176	190	14

Total Movement = 9 + 7 + 21 + 2 + 49 + 19 + 13 + 22 + 62 + 14 = 218 floors

➢ Solution B: Dual-Head SCAN (Zone Partitioning)



Zone Partition:

- Lower Zone (0-99): Requests = 79, 34, 60, 92, 11, 41, 13
- Upper Zone (100-200): Requests = 176, 114, 190

Head 1 (Lower Zone) - Starting at 50, sweeping DOWN:

Sort lower requests: 11, 13, 34, 41, 60, 79, 92

From	To	Movement
50	41	9
41	34	7
34	13	21
13	11	2

Head 1 Total = 39 floors

Head 2 (Upper Zone) - Starting at 150, sweeping UP:

Sort upper requests: 114, 176, 190

From	To	Movement
150	176	26
176	190	14

Head 2 Total = 40 floors

Result Comparison

System	Total Movement	Improvement
Single-Head SCAN	218 floors	–
Dual-Head SCAN	79 floors	**63.8% reduction**

❖ Code:

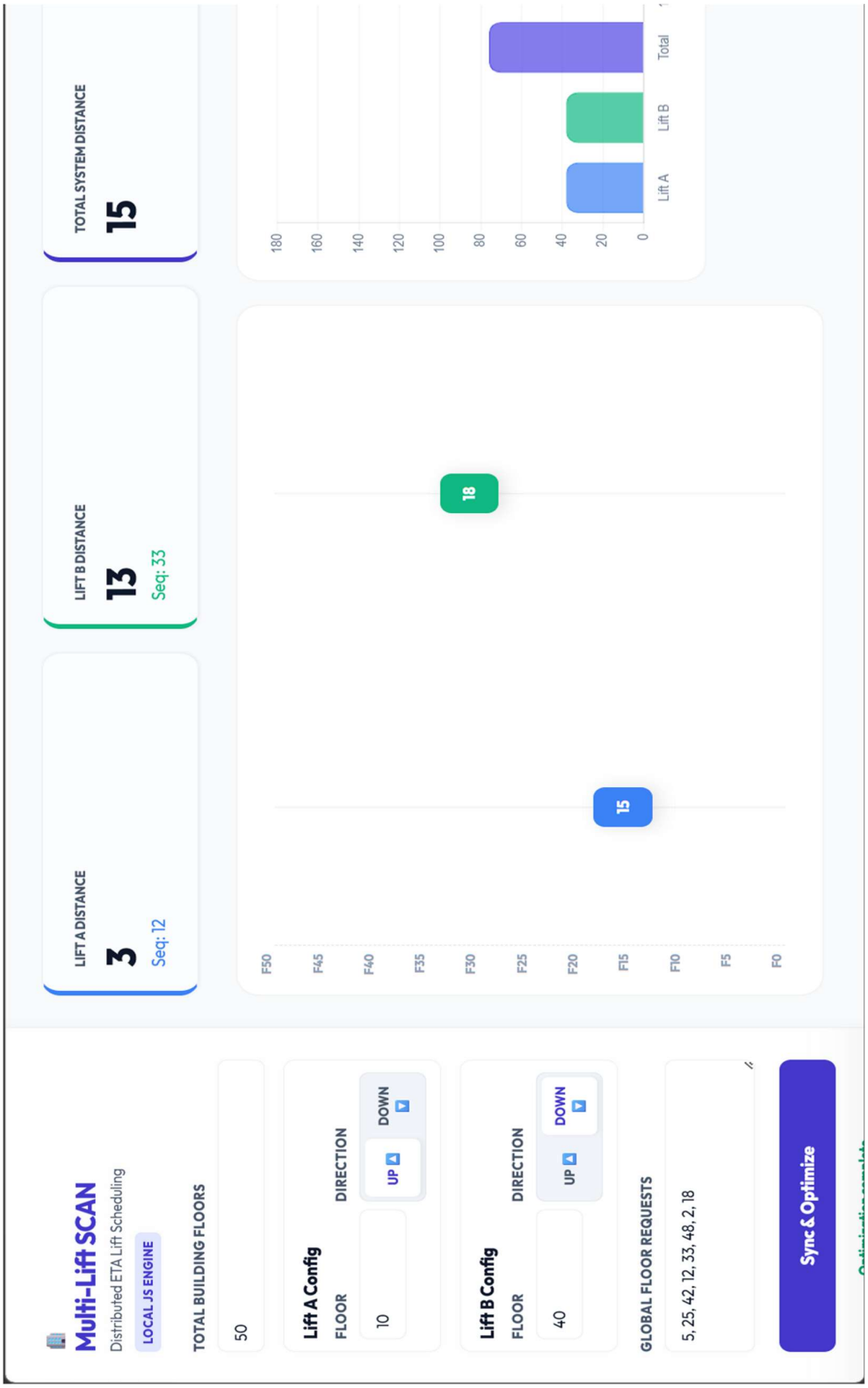
```
#include <stdio.h>
#include <stdlib.h>

/* --- COMPILE-TIME CONFIGURATION --- */
#define MAX_BUILDING_FLOOR 200
#define HEAD_1_START      50
#define HEAD_2_START      150

// The raw request sequence
static const int REQS[] = {176, 79, 34, 60, 92, 11, 41, 114, 13, 190};
#define N (sizeof(REQS) / sizeof(REQS[0]))

void sort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int t = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = t;
            }
        }
    }
}
```

```
int main() {
   int h1_reqs[N + 1], h2_reqs[N + 1];
   int h1_size = 0, h2_size = 0;

   printf("--- DUAL-HEAD BUILDING SCAN ---\n");
   printf("Max Building Height: %d floors\n", MAX_BUILDING_FLOOR);
   printf("Initial Positions  : Head 1 @ %d, Head 2 @ %d\n", HEAD_1_START,
HEAD_2_START);

   // Display the Request Sequence
   printf("Request Sequence   : ");
   for(int i = 0; i < N; i++) printf("%d%s", REQS[i], (i == N-1) ? "" : ", ");
   printf("\n\n----------------------------------------\n");

   // Partition requests: Head 1 takes lower half, Head 2 takes upper half
   for (int i = 0; i < N; i++) {
      if (REQS[i] < (MAX_BUILDING_FLOOR / 2)) {
         h1_reqs[h1_size++] = REQS[i];
      } else {
         h2_reqs[h2_size++] = REQS[i];
      }
   }

   // SCAN Requirement: Each head sweeps to its respective boundary
   h1_reqs[h1_size++] = 0;                // Head 1 sweeps to basement
   h2_reqs[h2_size++] = MAX_BUILDING_FLOOR; // Head 2 sweeps to roof

   sort(h1_reqs, h1_size);
   sort(h2_reqs, h2_size);

   // --- Head 1 Execution (Sweeping DOWN) ---
   int h1_pos = HEAD_1_START;
   int h1_total = 0;
   printf(">> HEAD 1 TRACE (Lower Zone: 0 - %d)\n",
(MAX_BUILDING_FLOOR/2)-1);
   for (int i = h1_size - 1; i >= 0; i--) {
      int dist = abs(h1_reqs[i] - h1_pos);
      h1_total += dist;
      printf("   Floor %3d -> %3d | Movement: %d\n", h1_pos, h1_reqs[i], dist);
      h1_pos = h1_reqs[i];
   }
   printf(">> HEAD 1 TOTAL MOVEMENT: %d floors\n\n", h1_total);

   // --- Head 2 Execution (Sweeping UP) ---
   int h2_pos = HEAD_2_START;
   int h2_total = 0;
```

```
    printf(">> HEAD 2 TRACE (Upper Zone: %d - %d)\n",
(MAX_BUILDING_FLOOR/2), MAX_BUILDING_FLOOR);
    for (int i = 0; i < h2_size; i++) {
        int dist = abs(h2_reqs[i] - h2_pos);
        h2_total += dist;
        printf("   Floor %3d -> %3d | Movement: %d\n", h2_pos, h2_reqs[i], dist);
        h2_pos = h2_reqs[i];
    }
    printf(">> HEAD 2 TOTAL MOVEMENT: %d floors\n", h2_total);

    printf("----------------------------------------\n");
    printf("COMBINED SYSTEM MOVEMENT: %d floors\n", h1_total + h2_total);
    printf("========================================\n");

    return 0;
}
```

❖ Output:

```
--- DUAL-HEAD BUILDING SCAN ---
Max Building Height: 200 floors
Initial Positions  : Head 1 @ 50, Head 2 @ 150
Request Sequence   : 176, 79, 34, 60, 92, 11, 41, 114, 13, 190

----------------------------------------
>> HEAD 1 TRACE (Lower Zone: 0 - 99)
   Floor  50 ->  92 | Movement: 42
   Floor  92 ->  79 | Movement: 13
   Floor  79 ->  60 | Movement: 19
   Floor  60 ->  41 | Movement: 19
   Floor  41 ->  34 | Movement: 7
   Floor  34 ->  13 | Movement: 21
   Floor  13 ->  11 | Movement: 2
   Floor  11 ->   0 | Movement: 11
>> HEAD 1 TOTAL MOVEMENT: 134 floors

>> HEAD 2 TRACE (Upper Zone: 100 - 200)
   Floor 150 -> 114 | Movement: 36
   Floor 114 -> 176 | Movement: 62
   Floor 176 -> 190 | Movement: 14
   Floor 190 -> 200 | Movement: 10
>> HEAD 2 TOTAL MOVEMENT: 122 floors
----------------------------------------
COMBINED SYSTEM MOVEMENT: 256 floors
========================================
```


Multi-Lift SCAN
Distributed ETA Lift Scheduling
LOCAL JS ENGINE
TOTAL BUILDING FLOORS
50
Lift A Config
FLOOR
DIRECTION
10
UP
DOWN
Lift B Config
FLOOR
DIRECTION
40
UP
DOWN
GLOBAL FLOOR REQUESTS
5, 25, 42, 12, 33, 48, 2, 18
Sync & Optimize
LIFT A DISTANCE
3
Seq: 12
LIFT B DISTANCE
13
Seq: 33
TOTAL SYSTEM DISTANCE
15
F50
F45
F40
F35
F30
F25
F20
F15
F10
F5
F0
15
18
180
160
140
120
100
80
60
40
20
0
Lift A
Lift B
Total

❖ Simulation QR:

❖ GitHub link: https://pranavcm24032.github.io/OS_POST_LAB/

❖ Conclusion:

The SCAN algorithm remains a cornerstone for smart building efficiency, effectively balancing energy conservation with passenger satisfaction. By streamlining movement patterns, it prevents excessive wait times for outlier requests. Furthermore, the integration of IoT sensor data offers a path to even higher optimization, allowing the system to dynamically adjust its range based on real-time floor occupancy.

❖ Discussion Question:

- **Does splitting the building exactly at the middle floor (100) cause one elevator to be overworked while the other sits idle?**
- **Why does Head 1 travel all the way to Floor 0 even if the lowest request is only at Floor 11?**
- **If Head 1 is sweeping DOWN but a new request appears at Floor 90, how long must that passenger wait?**
- **Is the extra movement to the boundary (Floor 0 or 200) worth the "predictability" of the SCAN algorithm?**
- **How does the code prevent both elevators from rushing to a request that appears exactly on the boundary (Floor 100)?**

Date : / / 2026

Signature

Course Co-Ordinator

B.Tech CSE(AIML)

Sem 4/2025-26